

Chapter 12

Treewidth and pathwidth of cocomparability graphs of bounded dimension

This paper gives a polynomial time algorithm for solving the treewidth and pathwidth problems on cocomparability graphs of any fixed dimension d .

The dimension of a partial order is one of the most carefully studied parameters of a partial order [165]. Yannakakis [174] showed that determining whether a partial order has dimension at most d is NP-complete for any fixed $d \geq 3$. Many problems have been shown to be efficiently solvable on partial orders of dimension 2. However no NP-complete partial order problem was known that is solvable by a polynomial time algorithm for partially ordered sets of some fixed dimension greater than 2.

In chapter 9 we have shown that treewidth and pathwidth can be solved in polynomial time for cocomparability graphs of fixed dimension d assuming that a d -dimensional representation is given as input. If the graph is given as input, this does not yield a polynomial time algorithm, since we cannot find the representation efficiently. Thus, it was unclear whether the problem was easy because the class of graphs is well behaved, or because having the representation (which is the solution to an NP-complete problem) is such a powerful tool that it gave the solution.

We show that treewidth and pathwidth are solvable on cocomparability graphs of fixed dimension in polynomial time, when the graph is given as input. This shows that small dimension is capturing a property which makes the problems simpler. Note that by Yannakakis' result [174], we are not able to run a recognition algorithm to determine whether our input is in the class of graphs we study here. However, this algorithm is designed so that if the input graph G is not a cocomparability graph the algorithm reports this, and otherwise it solves the treewidth and pathwidth problem for G correctly or reports that G has dimension larger than d . Thus, the result of

the algorithm can be used reliably even if we do not know *a priori* that G is in the class.

If the input G is not a cocomparability graph of dimension d , there are three possible outcomes. If G is not a cocomparability graph, which can be checked in time $O(n^{2.37\dots})$ [155], we reject the input. If G is a cocomparability graph, we can use the algorithm given in Theorem 11.4.3. (page 134) to either decide that the number of minimal separators is at larger than n^d , or listing all minimal separators. This algorithm can be implemented to run in time $O(n^{d+5})$. If the number of minimal separators is larger than n^d , we reject the input. If the number of minimal separators is at most n^d and G is a cocomparability graph, the algorithm computes the treewidth and pathwidth correctly, even if G has dimension greater than d .

This turns the TREEWIDTH and the PATHWIDTH problem into the first known problems for which restricted dimension helps, while both problems are NP-complete for cocomparability graphs when the dimension is unbounded [4].

12.1 Preliminaries

Recall Lemma 11.1.1, page 127: A subset S is a minimal separator if and only if there are two connected components such that every vertex of S has a neighbor in each.

The following lemma describes a useful property of minimal separators.

Lemma 12.1.1 *Let $G = (V, E)$ be a graph and S a minimal separator and a clique of G . Let C be a connected component of $G[V \setminus S]$ and let x and y be non adjacent vertices of $S \cup C$. Then every minimal x, y -separator S^* of G is a proper subset of $S \cup C$.*

Proof. Let S^* be a minimal x, y -separator of G and let C_x and C_y be the components of $G[V \setminus S^*]$ containing x and y , respectively.

C_x and C_y can not both have non-empty intersection with S since S is a clique. W.l.o.g. let $C_x \cap S = \emptyset$. Since S is a minimal separator of G , C_x is connected and $x \in C$ belongs to C_x , we get $C_x \subseteq C$. A vertex $s \in V \setminus (S \cup C)$ belongs to a component of $G[V \setminus S]$ different from C and can not have a neighbour in C_x . Hence, $s \notin S^*$. Finally, $S^* \subset S \cup C$ since $x, y \notin S^*$. $\square$

One of the main tools in our algorithm is the fact that all minimal separators of a graph can be computed in time polynomial times the number of minimal separators. Let R be the number of minimal separators of a graph G . Then there exists an algorithm which list all minimal separators in G in time $O(n^5 R)$. This was shown in chapter 11.